

Evidence-Gated Application Agent

Complete, verify, recover, and prove complex digital applications.

Core concept

An autonomous agent that converts a user’s documents and intent into a structured application workflow, maintains application state, verifies every critical action with machine-checkable evidence, safely recovers from failures and uncertainty, and never claims submission until the final state is independently proven.

A1 requirement	ROX interpretation
Execute multi-step tasks	Complete complex, multi-page, multi-document digital applications
Verify every action with machine-checkable evidence	Field provenance, document checks, portal state, upload acceptance, validation results, confirmation records
Detect failures	Missing/invalid documents, conflicting data, validation errors, upload failures, session expiry, unknown outcomes
Safely recover or re-plan	Failure-specific recovery, state reconstruction, bounded retries, alternate strategies, user escalation when needed
Never falsely claim completion	Only a deterministic Evidence Gate can transition the application to VERIFIED SUBMISSION

Working tagline: “RPA automates steps. ROX automates recovery from uncertainty.”

1. The Real-Life Problem

People regularly complete long digital applications that combine manual data entry, multiple documents, strict validation rules, uploads, payments, multi-page state, and final submission. Examples include scholarship applications, government hostel applications, certificates, bank/account opening, event registrations, competitions, and other institutional forms.

Why this is painful

The difficult part is not only typing. Users must figure out which document supports which field, reconcile conflicting information, meet portal constraints, recover from failed uploads or expired sessions, and determine whether a timed-out action actually succeeded.

2. Why “AI Form Filling” Is Not Enough

Weak framing	Why judges can reject it
OCR documents and extract fields	Document AI and OCR systems already do extraction and validation.
Auto-fill browser forms	Browser agents/RPA already automate navigation, field mapping, and submission.
Retry when something fails	A retry loop is not meaningful self-healing; blindly retrying can duplicate actions.
Show a dashboard of actions	A timeline alone is logging, not evidence-gated recovery.
Generate a preview before submit	Useful, but not unique; review-before-submit is common workflow design.

Design decision

ROX should not compete on OCR or browser automation. Those are tools. The product’s intelligence is state reconstruction, evidence-based action verification, safe recovery, and proof of final outcome.

3. Core Product Concept

ROX is an application-completion and recovery agent. The user provides the application goal and source documents. ROX determines what is required, builds an evidence map, proposes a plan, gets user approval, executes the workflow, verifies each critical step, and recovers when reality differs from the plan.



4. Core Differentiators

- 1. Application State Graph:** The agent maintains a machine-readable state of every important application component, including fields, documents, uploads, save state, and submission state.
- 2. Evidence-Backed Field Provenance:** Every critical value can be traced to a source document, page, extracted region/text, transformation, validation result, and confidence/consistency status.
- 3. Action Contracts:** Every high-impact action has explicit preconditions and postconditions. Example: after upload, the expected document must actually appear in the application.
- 4. Unknown-State Handling:** After a timeout, crash, or ambiguous response, ROX does not blindly repeat the action. It reconstructs the current state first.
- 5. Failure-Specific Recovery:** Different failure classes trigger different recovery strategies instead of generic retries.
- 6. Irreversible Action Guard:** Final submission, payment, cancellation, or other high-impact actions require stronger evidence and explicit human approval.
- 7. Portal Drift Adaptation:** If the portal changes fields, labels, constraints, or page structure, ROX detects the mismatch, re-plans, and re-verifies instead of silently using stale mappings.
- 8. Final Submission Proof:** A portal success message is not enough. ROX looks for independent confirmation such as application number, submitted status, timestamp, or receipt.
- 9. Recovery Resume:** After interruption, ROX resumes from the verified checkpoint instead of restarting the entire application.
- 10. Evidence Replay:** The user can inspect the complete execution and recovery trail, including failures, actions, evidence, and final proof.

5. End-to-End Example: Scholarship Application

User request: “Apply for this scholarship using my available documents.”

STEP 1 - INGEST DOCUMENTS

Aadhaar.pdf
10th_Marksheet.pdf
Income_Certificate.pdf
Bank_Passbook.pdf

STEP 2 - REQUIREMENT ANALYSIS

Personal details
Academic details
Income details
Bank details
Required certificates

STEP 3 - EVIDENCE MAP

Name → Aadhaar.pdf, page 1 ✓
DOB → Marksheet.pdf, page 1 ✓
Income → Income_Certificate.pdf, page 1 ✓
Account number → Bank_Passbook.pdf, page 1 ✓
Caste Certificate → MISSING △

STEP 4 - PLAN

Review → Fill → Upload → Validate → Submit

STEP 5 - USER APPROVAL

Approve / Modify

STEP 6 - EXECUTION

Field fill → evidence check

Document upload → postcondition check

STEP 7 - FAILURE

Income certificate upload failed: size limit exceeded

STEP 8 - RECOVERY

Compress document → verify readability → re-upload → verify acceptance

STEP 9 - SESSION EXPIRY

Reconstruct state → confirm saved fields → resume at last verified step

STEP 10 - FINAL SUBMISSION

User approves final preview → submit → verify application number + submitted state

RESULT

VERIFIED SUBMISSION

6. Failure Taxonomy and Recovery

Failure	Example	Recovery
Data failure	Missing / conflicting DOB	Cross-document reconciliation; user confirmation if unresolved
Document failure	Wrong type / unreadable PDF	Identify document; transform or request replacement
Constraint failure	File exceeds size limit	Compress/transform; validate before re-upload
Portal validation failure	Invalid field format	Interpret validation; transform value; re-verify
Session failure	Login/session expired	Re-authenticate through user boundary; reconstruct state; resume
Network failure	Request timeout	Reconcile actual portal state before repeating the action
Unknown state	Save/submit returned no response	Check current application state, timestamps, reference IDs, or receipt before any repeat
Portal drift	New required field or changed label	Detect schema/UI mismatch; rebuild field mapping and plan

Safety principle

Unknown state is not failure. Unknown state is a separate state that requires reconciliation before an action is repeated. This prevents duplicate payments, duplicate submissions, or destructive retries.

7. System Architecture



8. Evidence Model

Evidence should be a first-class object. Each important action produces structured evidence rather than an informal “success” message.

```
{
  "step_id": "UPLOAD_04",
  "action": "upload_income_certificate",
  "preconditions": ["file_readable", "file_size_valid"],
```

```
"command_or_action": "portal.upload(file)",
"observations": {
  "http_status": 200,
  "filename_visible": true
},
"postconditions": [
  "document_present_in_application"
],
"status": "VERIFIED",
"attempt": 2
}
```

9. Application State Machine



10. User Experience

The user interaction should remain simple. The complexity stays inside the agent and is exposed only when the user needs to review, resolve ambiguity, or approve a high-impact action.

ROX APPLICATION AGENT	
Goal: Apply for Scholarship	
32 fields verified	
4 documents verified	
1 conflict requires user input	

2 portal failures recovered	
STATUS: READY FOR FINAL REVIEW	
[View Evidence] [Review Application]	
[Approve & Submit]	

11. Evidence Dashboard

Metric	Meaning
Fields verified	Critical application fields backed by accepted evidence
Documents verified	Required documents identified, readable, valid, and accepted where applicable
Conflicts detected	Cases where sources disagree
Missing items	Required information/documents that cannot be proven
Failures detected	Execution failures encountered
Recovery attempts	Bounded recovery actions performed
State reconstructions	Times the system had to recover application state after interruption/uncertainty
Final checks passed	Acceptance conditions confirmed before completion
Unresolved issues	Anything preventing a verified final state

12. Initial Real-Life Use Cases

Use case	Representative workflow
Scholarship	Documents → form → validations → upload → review → submission proof
Government hostel	Personal/academic/income details → documents → eligibility fields → submission
Certificate application	Identity/address/income evidence → application → corrections → status/receipt
Account opening	Identity/address/bank information → document verification → form validation → review
Competition/event registration	Participant data → eligibility evidence → declarations → payment/registration confirmation

13. Why This Is Agentic, Not Just Automation

- It interprets a natural-language goal and derives the required workflow.
- It plans actions based on the current application state.
- It chooses among tools: document understanding, browser interaction, validation, transformation, state checks, and user input.
- It observes outcomes and compares them against explicit conditions.
- It diagnoses failure classes and selects recovery strategies.
- It can re-plan when the environment changes or an action enters an unknown state.
- It can stop and request human intervention when evidence is insufficient or an action is too risky.

14. Safety and Human-in-the-Loop Boundaries

Human approval is mandatory for

Final submission, payments, legal declarations, irreversible cancellations/withdrawals, and unresolved contradictions in personal/official information.

- Never guess a critical field when evidence is absent.
- Never resolve conflicting official data silently.
- Never repeat a high-impact action from an unknown outcome without reconciliation.
- Never claim completion because the LLM says the task is complete.
- Never expose sensitive document contents unnecessarily in logs or the dashboard.
- Use bounded recovery attempts and an explicit terminal failure state.

15. Hackathon MVP

For an overnight prototype, the full platform is unnecessary. Demonstrate one application type end-to-end with controlled simulated portal failures.

Component	MVP implementation
Input	User prompt + a small set of PDF/image documents
Document layer	OCR/document extraction + field provenance
Planner	Generate requirements, acceptance conditions, and plan draft
Approval	User approve/modify plan
Browser layer	Controlled demo web form
State engine	Track field/document/upload/submission state
Verifier	Deterministic checks for fields, uploads, portal state, and receipt
Recovery	3-5 failure types with bounded recovery strategies
Demo failures	Oversized upload, session expiry, validation error, conflicting field, submission timeout
Final output	Evidence-backed preview + final proof report

16. Recommended Demo Story

1. User uploads scholarship documents.
2. ROX extracts fields and shows source evidence.
3. ROX proposes a plan; user approves it.
4. ROX creates and starts filling the application.
5. A document upload deliberately fails because the file is too large.
6. ROX diagnoses the exact constraint, compresses the file, verifies readability, and re-uploads.
7. Session is deliberately expired.
8. ROX reconstructs application state and resumes from the last verified checkpoint.
9. A field conflict is detected between two documents.
10. ROX blocks autonomous completion and asks the user to resolve it.
11. User resolves the conflict; ROX continues.
12. Final submit is attempted.
13. Portal response becomes ambiguous / times out.
14. ROX does NOT click submit again; it reconciles application status.
15. Application number and submitted status are independently verified.
16. Dashboard shows the complete evidence + recovery history.

17. Final state: VERIFIED SUBMISSION.

17. Differentiation Summary

Typical form filler	ROX
Fills fields	Maintains verified application state
Extracts document data	Tracks provenance and evidence for critical values
Retries errors	Classifies failures and chooses recovery
Clicks submit	Guards irreversible actions
Assumes success	Reconciles unknown outcomes
Stops on interruption	Resumes from verified checkpoint
Shows logs	Produces an evidence-backed proof trail
Automates the happy path	Optimizes for recovery from the unhappy path

18. One-Minute Pitch

Pitch

“Digital applications are not simple forms. They are failure-prone transactions across documents, users, browsers, and backend systems. ROX is an evidence-gated application agent that understands the user’s goal, builds an execution plan, grounds critical fields in documents, verifies every important action, detects conflicts and uncertain outcomes, safely recovers from failures, resumes after interruptions, and refuses to claim completion until the final application state is independently proven. The browser is only a tool. The real product is verified recovery.”

19. Future Extensions

- More application domains and portal schemas
- API-level connectors where official APIs exist
- Adaptive portal schema learning
- User-owned reusable evidence vault with fine-grained privacy controls
- Automated correction suggestions for documents
- Cross-application reuse of verified profile information
- Stronger formal verification and audit trails
- Enterprise deployment for assisted digital service centers and help desks

20. Final Design Principle

Do not build another autofill bot.

Build an agent for the hardest part: safely moving a real digital application from an uncertain, partially completed state to a verified final state.